

LAB04 SIR discreto: ODE, Gillespie e tau-leaping

Antonio Scala

Obiettivi del laboratorio

In questo laboratorio studieremo lo stesso modello epidemico SIR a tre livelli di descrizione:

1. tramite equazioni differenziali ordinarie per la dinamica media;
2. tramite simulazione esatta a eventi discreti con algoritmo di Gillespie;
3. tramite approssimazione tau-leaping.

L'obiettivo non è solo implementare tre codici diversi, ma capire:

- come si passa da una dinamica discreta e stocastica a una descrizione media continua;
- dove compare un'approssimazione di **chiusura**;
- perché una singola realizzazione stocastica può differire sensibilmente dalla dinamica media;
- quando il tau-leaping accelera bene la simulazione;
- quando invece fallisce o introduce errori visibili;
- in quali regimi conviene usare direttamente Gillespie.

Struttura del laboratorio

Il laboratorio è diviso in cinque parti:

- Parte 0 – richiamo teorico: dal SIR discreto all'ODE media;
- Parte A – implementazione Gillespie;
- Parte B – confronto tra traiettorie stocastiche e ODE;
- Parte C – tau-leaping;
- Parte D – confronto finale tra i tre livelli di descrizione;
- Appendice – note pratiche su Python, integratori e numeri casuali.

Parte 0 – Richiamo teorico

0.1 Modello SIR a eventi discreti

Consideriamo una popolazione chiusa di taglia totale costante

$$N = S + I + R.$$

Gli eventi possibili sono due:

1. **infezione**

$$(S, I, R) \rightarrow (S - 1, I + 1, R)$$

con tasso

$$a_1(S, I, R) = \beta \frac{SI}{N};$$

2. **guarigione**

$$(S, I, R) \rightarrow (S, I - 1, R + 1)$$

con tasso

$$a_2(S, I, R) = \gamma I.$$

Questo è il modello di partenza sia per Gillespie sia per tau-leaping.

0.2 Equazioni per i valori medi

Se denotiamo con $\langle S \rangle$, $\langle I \rangle$, $\langle R \rangle$ i valori medi sulle realizzazioni del processo, dalle regole di transizione si ottiene formalmente

$$\frac{d}{dt} \langle S \rangle = -\beta \frac{\langle SI \rangle}{N},$$

$$\frac{d}{dt} \langle I \rangle = \beta \frac{\langle SI \rangle}{N} - \gamma \langle I \rangle,$$

$$\frac{d}{dt} \langle R \rangle = \gamma \langle I \rangle.$$

Il punto importante è che compare la quantità

$$\langle SI \rangle,$$

cioè una media del prodotto, non semplicemente il prodotto delle medie.

0.3 Chiusura media-field

Per ottenere il sistema ODE standard si introduce una chiusura semplice:

$$\langle SI \rangle \approx \langle S \rangle \langle I \rangle.$$

In parole semplici: si trascura il fatto che S e I possano essere correlati. Con questa approssimazione si ottiene il sistema deterministico usuale:

$$\dot{S} = -\beta \frac{SI}{N},$$

$$\dot{I} = \beta \frac{SI}{N} - \gamma I,$$

$$\dot{R} = \gamma I.$$

Questa è la prima grande approssimazione del laboratorio: una dinamica stocastica a eventi discreti viene sostituita da una dinamica continua e deterministica.

0.4 Domande guida

1. Perché nelle equazioni medie compare $\langle SI \rangle$ e non direttamente $\langle S \rangle \langle I \rangle$?
2. In quali situazioni la chiusura media-field potrebbe essere ragionevole?
3. In quali situazioni potrebbe essere fuorviante?

Parte A – Gillespie per il modello SIR

A1. Idea dell'algoritmo

Nel metodo di Gillespie il tempo non avanza con passi fissi.

Ad ogni iterazione:

1. si calcolano i tassi

$$a_1 = \beta \frac{SI}{N}, \quad a_2 = \gamma I;$$

2. si sommano:

$$a_0 = a_1 + a_2;$$

3. si estrae il tempo del prossimo evento:

$$\tau \sim \text{Exp}(a_0);$$

4. si sceglie quale evento avviene con probabilità

$$P(\text{infezione}) = \frac{a_1}{a_0}, \quad P(\text{guarigione}) = \frac{a_2}{a_0};$$

5. si aggiorna lo stato.

A2. Implementazione

Usare stato

$$x = [S, I, R]$$

e variazioni di stato

$$\nu_1 = [-1, +1, 0], \quad \nu_2 = [0, -1, +1].$$

Uno schema quasi-pseudocodice è:

```
while I > 0 and t < T:
    a1 = beta * S * I / N
    a2 = gamma * I
    a0 = a1 + a2

    tau = exponential(mean = 1/a0)

    scegli evento:
        infezione con probabilità a1/a0
        guarigione con probabilità a2/a0

    aggiorna S, I, R
    aggiorna t = t + tau
```

A3. Parametri suggeriti

Come primo test, usare ad esempio

$$N = 200, \quad S_0 = 199, \quad I_0 = 1, \quad R_0 = 0,$$

$$\beta = 0.5, \quad \gamma = 0.2.$$

In seguito conviene esplorare anche altri regimi.

A4. Compiti

1. Simulare una singola traiettoria fino a un tempo finale T .
2. Tracciare $S(t)$, $I(t)$, $R(t)$.
3. Ripetere più volte con lo stesso stato iniziale ma seed diversi.

A5. Cosa osservare

1. L'epidemia parte sempre?
2. Il tempo iniziale di crescita di $I(t)$ è sempre lo stesso?
3. Una singola traiettoria assomiglia alla soluzione ODE oppure no?
4. Quanto sono importanti le fluttuazioni quando il numero di infetti è piccolo?

Parte B – Confronto tra Gillespie e ODE

B1. Integrazione dell'ODE

Integrare numericamente il sistema deterministico SIR con gli stessi parametri iniziali:

$$\dot{S} = -\beta \frac{SI}{N}, \quad \dot{I} = \beta \frac{SI}{N} - \gamma I, \quad \dot{R} = \gamma I.$$

Si può usare:

- un semplice schema di Eulero esplicito, se il passo è sufficientemente piccolo;
- oppure `solve_ivp` di SciPy, che è più robusto e gestisce automaticamente il passo (vedi Appendice).

B2. Perché SciPy e non NumPy?

In Python è importante distinguere tra:

- **NumPy**, che fornisce array, algebra lineare di base e generatori di numeri casuali;
- **SciPy**, che aggiunge strumenti scientifici più avanzati, tra cui gli integratori ODE.

Quindi:

- per vettori, griglie temporali e random si usa soprattutto **NumPy**;
- per integrare un sistema ODE si usa tipicamente **SciPy**.

Vantaggi di `solve_ivp`

- gestisce direttamente sistemi di equazioni;
- sceglie automaticamente il passo;
- è più robusto di uno schema scritto a mano;
- restituisce i risultati in forma già comoda per i grafici.

Svantaggi didattici

- è più “scatola nera” di Eulero;
- nasconde la logica della discretizzazione.

Per questo, a livello concettuale, è utile ricordare che anche `solve_ivp` sta sempre eseguendo un’integrazione numerica discreta.

B3. Confronto qualitativo

Sovrapporre ad una singola traiettoria Gillespie la soluzione ODE.

Domande guida

1. L’accordo è buono all’inizio? al picco? in coda?
2. Una singola traiettoria è il confronto giusto per l’ODE?
3. In che senso l’ODE rappresenta una dinamica “media”?

B4. Media su repliche

Generare molte traiettorie Gillespie indipendenti, ad esempio $M = 50$ oppure $M = 100$, e costruire una media empirica di $S(t)$, $I(t)$, $R(t)$ su una griglia temporale comune.

Compiti

1. Interpolare le traiettorie su una griglia uniforme in tempo.
2. Calcolare le medie empiriche.
3. Confrontare media Gillespie e soluzione ODE.

Domande guida

1. La media delle traiettorie è più vicina all’ODE di quanto lo sia una singola traiettoria?
2. Dove le differenze restano più visibili?
3. Che ruolo ha la taglia della popolazione N in questo confronto?

Parte C – Tau-leaping

C1. Idea

Nel tau-leaping si sceglie un passo temporale Δt e si assume che, in quell’intervallo, i tassi restino quasi costanti.

Il numero di eventi di ciascun tipo viene approssimato con variabili di Poisson:

$$K_1 \sim \text{Poisson}(a_1(x)\Delta t), \quad K_2 \sim \text{Poisson}(a_2(x)\Delta t).$$

L'aggiornamento è allora

$$x \rightarrow x + K_1\nu_1 + K_2\nu_2.$$

Uno schema quasi-pseudocodice è:

```
while I > 0 and t < T:
    a1 = beta * S * I / N
    a2 = gamma * I

    k1 = poisson(a1 * dt)
    k2 = poisson(a2 * dt)

    S = S - k1
    I = I + k1 - k2
    R = R + k2

    t = t + dt
```

C2. Avvertenza importante

Il tau-leaping è un'approssimazione. Può funzionare bene quando in un intervallo Δt avvengono molti eventi ma i tassi non cambiano troppo.

Può invece fallire quando:

- i numeri sono piccoli;
- l'epidemia è appena partita;
- un singolo evento cambia in modo importante i tassi;
- il passo Δt è troppo grande.

Con passi troppo grandi possono comparire problemi non fisici, ad esempio:

- popolazioni negative;
- salti troppo bruschi;
- perdita della fase iniziale corretta;
- picchi spostati o distorti.

Nel laboratorio, se compaiono stati impossibili, non è necessario progettare subito un metodo sofisticato di correzione: basta segnalarli e discuterne.

C3. Esperimento numerico

Confrontare almeno tre valori di Δt , ad esempio

$$\Delta t = 0.01, \quad 0.1, \quad 0.5$$

oppure una griglia simile.

Per ciascun valore:

1. simulare una traiettoria;
2. confrontarla con Gillespie;
3. confrontarla con l'ODE.

C4. Domande guida

1. Per quali Δt il tau-leaping riproduce bene Gillespie?
2. Quando inizia a perdere la fase iniziale corretta?
3. Quando produce salti poco plausibili?
4. Compaiono stati non fisici?
5. In quali regimi accelera molto il calcolo senza degradare troppo l'accuratezza?

Parte D – Dove funzionano le approssimazioni?

D1. Confronto finale

Riassumere il comportamento dei tre approcci.

ODE

- molto economica;
- descrive la dinamica media;
- non vede bene la variabilità delle traiettorie;
- non cattura l'estinzione precoce di una piccola infezione.

Gillespie

- è la simulazione esatta del processo a eventi discreti;
- cattura bene fluttuazioni, ritardi e mancata partenza dell'epidemia;
- è più costosa quando il numero di eventi è molto grande.

Tau-leaping

- è un'approssimazione veloce di Gillespie;
- funziona bene quando i tassi cambiano poco nel salto;
- può fallire quando i numeri sono piccoli o la dinamica cambia rapidamente.

D2. Esperimenti suggeriti

Ripetere il confronto in almeno due regimi distinti.

Regime 1 – popolazione relativamente piccola

$$N \sim 10^2$$

Regime 2 – popolazione più grande

$$N \sim 10^3 \text{ oppure } 10^4$$

e discutere come cambia la qualità delle approssimazioni.

D3. Domande finali

1. Quando la soluzione ODE è una buona descrizione?
2. Quando la dinamica stocastica mostra fenomeni che l'ODE non può vedere?
3. Quando il tau-leaping è affidabile?
4. Quando vale la pena pagare il costo di Gillespie?

Materiale di partenza

Per il laboratorio si suggerisce di partire da tre script:

- 01_sir_ode.py
- 02_sir_gillespie.py
- 03_sir_tau_leaping.py

ed eventualmente da un quarto file di confronto:

- 04_compare_methods.py

Consegna

Ogni gruppo deve produrre:

1. una derivazione breve delle equazioni medie del SIR con spiegazione della chiusura;
2. almeno una figura con traiettoria Gillespie e soluzione ODE sovrapposte;
3. una figura con media su molte traiettorie Gillespie e soluzione ODE;
4. un confronto tra tau-leaping e Gillespie per almeno tre valori di Δt ;
5. un breve commento finale su:
 - differenza tra dinamica media e dinamica stocastica;
 - casi in cui l'epidemia non parte;
 - quando tau-leaping funziona o fallisce;
 - quando conviene usare Gillespie.

Checklist finale

1. Derivazione delle equazioni medie.
2. Chiusura $\langle SI \rangle \approx \langle S \rangle \langle I \rangle$ discussa esplicitamente.
3. Implementazione Gillespie del SIR.
4. Confronto con ODE su singola traiettoria.
5. Media su repliche.
6. Implementazione tau-leaping.
7. Discussione sui regimi di validità dei tre metodi.

Appendice — Note pratiche su implementazione

Questa appendice raccoglie alcune indicazioni pratiche per l'implementazione numerica.

Non è necessario leggerla tutta prima di iniziare il laboratorio: può essere usata come riferimento durante lo svolgimento.

A.1 Perché inserire piccoli snippet Python?

In questo laboratorio useremo Python per produrre rapidamente simulazioni e grafici, ma la logica degli algoritmi non dipende da Python.

Per questo motivo, accanto ai passaggi principali conviene distinguere tra:

- **logica dell'algoritmo;**
- **implementazione concreta** in un certo linguaggio.

Uno snippet Python molto breve può essere letto come una forma di pseudocodice operativo: non è solo “codice da eseguire”, ma anche una descrizione compatta della sequenza di operazioni.

Questo è utile anche a chi usa altri linguaggi, perché la traduzione in Matlab, R o Julia è in genere immediata.

A.2 Integrare un sistema ODE con `solve_ivp`

Supponiamo di voler integrare

$$\dot{x} = f(t, x)$$

con condizione iniziale $x(0) = x_0$.

In Python la struttura tipica è:

```
from scipy.integrate import solve_ivp
import numpy as np

def sir_rhs(t, y, beta, gamma, N):
    S, I, R = y
    dS = -beta * S * I / N
    dI = beta * S * I / N - gamma * I
    dR = gamma * I
    return [dS, dI, dR]

t0 = 0.0
T = 60.0
y0 = [199, 1, 0]
t_grid = np.linspace(t0, T, 400)

sol = solve_ivp(
    sir_rhs,
    [t0, T],
    y0,
    t_eval=t_grid,
    args=(beta, gamma, N)
)
```

Come si legge

- `sir_rhs` è la funzione che definisce il lato destro dell'ODE;
- `[t0, T]` è l'intervallo temporale;

- `y0` è lo stato iniziale;
- `t_eval=t_grid` dice all'integratore in quali tempi vogliamo la soluzione;
- `args=(beta, gamma, N)` passa i parametri extra alla funzione.

Cosa restituisce

L'oggetto `sol` contiene soprattutto:

- `sol.t`: i tempi;
- `sol.y`: la soluzione; ogni riga corrisponde a una componente del vettore di stato.

Ad esempio:

```
S = sol.y[0]
I = sol.y[1]
R = sol.y[2]
```

A.3 E se non uso Python?

La struttura concettuale è la stessa anche in altri linguaggi.

Matlab

In Matlab l'integratore standard per ODE non stiff è tipicamente `ode45`.

Schema tipico:

```
[t,y] = ode45(@(t,y) sir_rhs(t,y,beta,gamma,N), [t0 T], y0);
```

R

In R per le ODE si usa spesso il pacchetto `deSolve`, con la funzione `ode`.

Schema tipico:

```
library(deSolve)
out <- ode(y = y0, times = times, func = sir_rhs, parms = pars)
```

Quindi: cambiano i nomi delle funzioni, ma non cambia la logica.

A.4 Numeri casuali in Python

Per simulazioni stocastiche è consigliato usare il generatore moderno di NumPy:

```
import numpy as np
rng = np.random.default_rng(12345)
```

Qui 12345 è il `seed`, cioè il valore iniziale che rende la simulazione riproducibile.

Uniforme in $[0, 1)$

```
u = rng.random()
```

Esponenziale di media scale

```
tau = rng.exponential(scale=1/a0)
```

Poisson di media lam

```
k = rng.poisson(lam=a0 * dt)
```

Perché usare un oggetto rng

È preferibile a funzioni globali del tipo `np.random.rand()` perché:

- rende il codice più controllabile;
- facilita la riproducibilità;
- evita confusione quando si confrontano metodi diversi.

A.5 Numeri casuali in altri linguaggi

Matlab

- uniforme: `rand`
- Poisson: `poissrnd`
- esponenziale: con funzioni statistiche dedicate, oppure via inversione

R

- uniforme: `runif`
- esponenziale: `rexp`
- Poisson: `rpois`

Anche qui la struttura resta la stessa:

- si definisce il modello;
- si aggiornano i dati nel tempo;
- si usano generatori casuali per la parte stocastica.

A.6 Morale pratica

Cambiano i nomi delle funzioni, ma non cambia la struttura concettuale:

1. definisco lo stato del sistema;

2. definisco la dinamica deterministica o stocastica;
3. aggiorno nel tempo;
4. memorizzo i risultati;
5. confronto i metodi.

Per questo, capire bene l'algoritmo è più importante che ricordare la sintassi di un singolo linguaggio.